SIAE & DataPizza Hackathon 2026

Build the Plugin. Use the Plugin. Ship the Product.

1. La Visione

Claude Code è un tool potentissimo, ma senza guida strutturata tende a saltare fasi critiche: scrive codice prima dei test, committa messaggi non convenzionali, dichiara “fatto” senza prove. Il contesto riparte da zero a ogni sessione e la qualità dipende interamente dalla disciplina del developer.

Un **plugin per Claude Code** risolve questo alla radice. Non è un’applicazione con una UI. È un sistema di **hook**, **skill** e **agent** che lavora dietro le quinte: gli hook intercettano i momenti critici, le skill forniscono le istruzioni operative, gli agent delegano compiti a sotto-sessioni controllate.

Oggi costruirai il tuo plugin. Lo pubblicherai su GitHub. E dovrà essere installabile da chiunque con un singolo comando, senza errori.

⚠ Regole della prova

- Hai 8 ore. Plugin e prodotto pesano uguale: gestisci il tempo di conseguenza.
- Max 800 righe di codice tra hook, script e agent. I file di test e le skill (Markdown) non contano.
- Il plugin deve essere installabile e funzionante su una macchina pulita.
- Ogni agent deve dichiarare un budget di token nel frontmatter.
- Almeno 6 test case funzionanti sugli hook/script. Per il frontend, 70% coverage con vitest.

Story 1 + Story 2 = un unico progetto

- Story 1: costruisci il plugin. Story 2: SIAE+
- Il plugin senza il prodotto non dimostra che funziona.
- Il prodotto senza il plugin non dimostra che sei stato guidato.
- La valutazione è sull’insieme.

2. Prima di Scrivere una Riga

Non partire dal codice. Parti dalla documentazione. Leggi questi riferimenti obbligatori:

- **Claude Code Hooks:** https://code.claude.com/docs/en/hooks-guide
- **Claude Code Skills:** https://code.claude.com/docs/en/skills
- **Superpowers (Jesse Vincent):** https://github.com/obra/superpowers — il framework di riferimento
- **Plugin README Anthropic:** https://github.com/anthropics/claude-code/blob/main/plugins/README.md

Non devi copiare Superpowers. Devi **capire i pattern** (come sono strutturate le skill, come funzionano gli hook, come si orchestra il tutto) e **adattarli al tuo plugin**.

3. Story 1 — Il Plugin

3.1 Struttura del Repository

Il tuo plugin deve seguire la struttura standard di un plugin Claude Code. Ecco lo scheletro minimo:

```
mio-plugin/
├── .claude-plugin/
│   └── plugin.json        ← metadata del plugin
├── hooks/
│   └── hooks.json         ← configurazione hook
├── skills/
│   ├── brainstorming/
│   │   └── SKILL.md       ← analisi prima di scrivere codice
│   ├── implementation/
│   │   └── SKILL.md       ← convenzioni e standard di codice
│   ├── tdd/
│   │   └── SKILL.md       ← ciclo RED → GREEN → REFACTOR
│   └── verification/
│       └── SKILL.md       ← gate finale prima di dichiarare "fatto"
├── agents/
│   └── reviewer.md        ← almeno 1 agent
├── scripts/
│   └── *.sh               ← script richiamati dagli hook
├── tests/
│   └── *.test.sh          ← test degli hook/script
└── README.md
```

Sei libero di aggiungere skill, agent o hook. Lo scheletro sopra è il minimo per il livello Base.

3.2 Come Funziona — Il Quadro d'Insieme

Ecco cosa succede durante una sessione Claude Code con il tuo plugin attivo. Ogni riga è un momento del ciclo di sviluppo, con il componente del plugin che interviene:

Momento	Cosa succede	Componente	Effetto
① Avvio sessione	L'utente apre Claude Code nel progetto	SessionStart hook	Inietta contesto: lista skill, agent, regole attive
② Nuova feature	L'utente chiede "implementa X"	brainstorming skill	Forza analisi: problema, casi limite, approcci
③ Scrittura codice	Claude sta per creare/editare un file	tdd skill + TDD gate hook	Impone: scrivi il test prima del codice
④ Implementazione	Claude scrive il codice di produzione	implementation skill	Applica convenzioni: naming, struttura, import
⑤ Commit	Claude lancia git commit -m "..."	PreToolUse(Bash) hook	Valida formato Conventional Commits. Blocca se invalido
⑥ Ogni operazione	Claude completa un tool use qualsiasi	PostToolUse hook	Logga su .session-log.jsonl (prova per la giuria)
⑦ "Ho finito"	L'utente o Claude dichiarano task completo	verification skill	Gate: verifica prove concrete prima di confermare
⑧ Review	Serve una revisione del codice	reviewer agent	Subagent dedicato: analizza, produce report strutturato

Tabella formattata

3.3 Gli Hook — Il Sistema Nervoso

Gli hook sono script o comandi che Claude Code esegue automaticamente in risposta a eventi del suo ciclo di vita. Non aspettano input dell'utente: agiscono prima o dopo che Claude fa qualcosa. La configurazione va in hooks/hooks.json dentro il plugin.

Come funzionano gli hook (in 30 secondi)

- SessionStart → si attiva quando apri una sessione Claude Code.
- PreToolUse → si attiva PRIMA che Claude usi un tool (Bash, Write, Edit).
- PostToolUse → si attiva DOPO che Claude ha usato un tool.
- Exit code 0 = tutto ok, lascia procedere.
- Exit code 2 = BLOCCA l'azione e manda il messaggio di stderr a Claude.
- L'input arriva come JSON su stdin (tool_name, tool_input, ecc.).

Tabella formattata

Hook obbligatori (minimo 2)

Devi implementare almeno 2 hook funzionanti. Ecco quelli consigliati, in ordine di priorità:

1. SessionStart hook — Inietta contesto all'avvio

Questo è lo hook più importante. Si attiva automaticamente quando apri una nuova sessione Claude Code nella directory del progetto. Il suo compito è concreto:

- Scansiona la cartella skills/ del plugin e legge il frontmatter YAML di ogni SKILL.md (nome, tipo, trigger).
- Scansiona la cartella agents/ e raccoglie i nomi degli agent disponibili.
- Stampa su stdout un riassunto strutturato con: lista skill, lista agent, regole attive.
- Claude Code cattura questo stdout e lo usa come contesto persistente per tutta la sessione.

Risultato pratico: senza questo hook, Claude non sa che il plugin esiste. Con questo hook, Claude sa fin dal primo messaggio quali skill invocare, quali agent delegare, e quali regole rispettare. È il punto di ingresso di tutto il sistema.

2. PreToolUse(Bash) hook — Quality gate sui commit

Ogni volta che Claude sta per lanciare un comando bash, questo hook controlla se il comando contiene git commit. Se sì, valida il messaggio contro il formato Conventional Commits. Se non è valido, exit 2 + messaggio d'errore su stderr.

3. PreToolUse(Write|Edit) hook (opzionale) — TDD gate

Prima che Claude scriva o modifichi un file di codice, controlla che esista già un file di test corrispondente. Se non esiste, blocca e chiede di scrivere prima il test. Non si applica ai file di test stessi né ai file di configurazione.

4. PostToolUse hook (opzionale) — Session log

Dopo ogni operazione, logga su un file .session-log.jsonl il timestamp, il tool usato, il file coinvolto e l'esito. È la prova concreta che il plugin era attivo durante lo sviluppo.

3.4 Le Skill — Le Istruzioni Operative

Le skill sono file Markdown con un **frontmatter YAML** che Claude legge per decidere quando attivarle. Non sono documentazione: sono **istruzioni operative** che Claude esegue passo per passo.

Ogni skill vive in una sottocartella di skills/ e si chiama SKILL.md. La struttura interna è sempre:

1. Frontmatter YAML: name, description (con TRIGGER conditions esplicite)
2. Banner identificativo: riga visuale che mostra il nome quando viene caricata
3. Step numerati: azioni concrete che Claude esegue in sequenza
4. Label di rischio: LOW, MEDIUM o HIGH

Skill obbligatorie (minimo 4)

1. brainstorming (analisi prima del codice)

Si attiva prima di qualsiasi implementazione non banale. Forza una fase di analisi: definisci il problema, identifica i casi limite, valuta approcci alternativi, stima la complessità. Intercetta frasi come “implementa”, “scrivi”, “crea”, “aggiungi una feature”.

2. implementation (convenzioni di codice)

Si attiva quando Claude sta per scrivere codice. Definisce le convenzioni concrete per lo stack scelto: naming (snake_case per funzioni Python, PascalCase per classi), struttura file, lunghezza massima funzioni, regole import. Non scrivere regole vaghe come “usa nomi significativi”: scrivi regole specifiche.

3. tdd (test-driven development)

Si attiva quando Claude deve implementare una funzione, un metodo o un componente. Impone il ciclo RED → GREEN → REFACTOR: prima scrivi il test e verifica che fallisca (rosso), poi scrivi il codice minimo per farlo passare (verde), poi rifattorizza. Gli step devono essere sequenziali e non saltabili.

4. verification (gate finale)

Si attiva quando Claude o l’utente dichiarano un task completato. Intercetta frasi come “è fatto”, “ho finito”, “done”, “posso fare il merge”. Impone una sequenza fissa: identifica cosa deve essere vero, esegui i comandi di verifica, leggi l’output, verifica che ogni punto sia verde. Se anche uno solo non è verde, si torna indietro.

Skill aggiuntive suggerite (per punteggio più alto):

- git-workflow — naming branch, formato commit, checklist pre-push
- documentation — README, note architetturali, istruzioni di setup
- ci-cd (bonus) — guida Claude nella configurazione di pipeline CI/CD e GitHub Actions

3.5 Gli Agent — Delegazione Controllata

Un agent è un file Markdown in agents/ con un **system prompt dedicato**. Claude può delegare compiti a un agent che lavora in una sotto-sessione con contesto separato. Questo evita di inquinare il contesto principale.

Regola critica: budget di token

Ogni agent DEVE avere un budget esplicito nel frontmatter (es. max_tokens: 8000).
Senza budget, un agent può consumare tutto il contesto disponibile.
Tieni i budget bassi: un reviewer non ha bisogno di 100k token.

Tabella formattata

Agent obbligatorio (minimo 1)

reviewer (esempio di agent)

Riceve un file o un diff e restituisce un report strutturato: problemi trovati, suggerimenti, severity (CRITICAL/WARNING/INFO). Claude lo invoca automaticamente prima di dichiarare un task completato, oppure l'utente lo chiama esplicitamente.

Altri agent suggeriti:

- test-writer — genera test a partire da un'interfaccia o specifica
- doc-writer — genera documentazione a partire dal codice

3.6 Il Tuo Strumento — Usa Claude Code

Usa Claude Code per sviluppare il plugin stesso. Non devi scrivere tutto a mano: apri Claude Code nella directory del tuo plugin e lavora insieme a lui. Digli cosa vuoi fare ("crea un hook SessionStart che scansiona le skill") e lascia che ti aiuti a implementarlo.

Questo ha un doppio vantaggio: sviluppi più velocemente e testi i tuoi hook in tempo reale. Man mano che il plugin prende forma, vedrai Claude Code reagire ai tuoi hook bloccare commit non validi, iniettare il contesto, invocare le skill.

ha eliminato: —

Flusso pratico

1. Apri Claude Code nella directory del plugin.
2. Chiedi: "Crea lo scheletro del plugin con plugin.json, hooks.json, e la prima skill."
3. Implementa il primo hook (SessionStart). Riavvia la sessione e verifica che il contesto appaia.
4. Implementa il secondo hook (PreToolUse). Prova un commit sbagliato e verifica il blocco.
5. Aggiungi skill e agent. Claude li vedrà automaticamente al prossimo avvio sessione.
6. Quando il plugin funziona, usalo per la Story 2: ogni azione passerà attraverso i tuoi hook.

3.7 Timeline — Come Distribuire le 4 Ore per costruire il plugin

Ecco la distribuzione consigliata del tempo. Non è obbligatoria, ma è pensata per massimizzare il risultato.

Fase 1 — Setup (30 min)

5. Crea o fai creare a Claude Code il repo con la struttura base: .claude-plugin/plugin.json, hooks/, skills/, agents/, tests/
6. Leggi la documentazione. Non saltare questo passaggio.
7. Inizializza git e fai il primo commit: chore: scaffold plugin structure

Fase 2 — Primo Hook + Prime 2 Skill (60 min)

8. Implementa il SessionStart hook: scansiona skills/ e agents/, stampa il contesto su stdout.

9. Implementa le skill brainstorming e implementation: frontmatter con trigger espliciti, step operativi concreti.
10. Testa manualmente: apri Claude Code nella directory e verifica che il contesto venga iniettato.
11. Scrivi almeno 1 test per l'hook.
12. Commit: feat(hooks): add session-start hook

Fase 3 — Secondo Hook + Agent + Skill 3-4 (60 min)

13. Implementa il PreToolUse(Bash) hook per validare i commit (Conventional Commits).
14. Implementa l'agent reviewer: system prompt + frontmatter con budget.
15. Implementa le skill tdd e verification.
16. Testa: prova un commit con messaggio sbagliato e verifica il blocco.
17. Commit: feat(hooks): add pre-commit conventional commits gate

Fase 4 — Completamento + Test (60 min)

18. Aggiungi hook opzionali se hai tempo (TDD gate, post-tool logger).
19. Aggiungi skill bonus (git-workflow, documentation, ci-cd).
20. Porta la test coverage al 70%.
21. Verifica l'installabilità: claude plugin add --path ./tuo-plugin

Fase 5 — README + Pubblicazione (30 min)

22. Scrivi il README: cosa fa il plugin, come si installa, come si usa, perché delle scelte.
23. Pubblica su GitHub. Verifica che un clone pulito funzioni.
24. Se hai tempo: aggiungi una GitHub Action per test automatici su push/PR.

💡 **Consigli pratici**

- Usa Claude Code per tutto: scaffolding, implementazione, test, debug. È il tuo copilota.
- Non cercare la perfezione. Un hook che funziona vale 10 hook che stampano solo messaggi.
- Testa spesso. Ogni 20 min, riavvia la sessione e prova il plugin in azione.
- Fai commit frequenti con Conventional Commits: la storia git è parte della valutazione.
- Se sei bloccato su un hook, passa alla skill o all'agent. Non restare fermo.
- Chiedi a Claude Code: "Mostrami cosa hai capito del plugin" per verificare che il contesto sia iniettato.

4. Story 2 — Il Prodotto

4.1 Il contesto applicativo — L'app SIAE+

SIAE+ è l'applicazione mobile a disposizione per gli autori ed editori associati a SIAE per la verifica delle proprie performance, l'inserimento di depositi delle proprie opere, la consultazione dei pagamenti ricevuti e tanto altro.

L'app è disponibile esclusivamente per dispositivi mobili, Android e iOS, ed è sviluppata sulla base di un design system e di regole di UX/UI che rappresentano lo stile ed il brand SIAE più recenti.

Il design system può essere importato all'interno dell'app attraverso la libreria @itsiae/siae-design-system messa a disposizione.

Il prodotto da consegnare dovrà essere coerente con lo stile ed il design dell'app SIAE+.

4.2 La richiesta — Requisiti funzionali

Il business SIAE ha richiesto un aggiornamento all'attuale processo di iscrizione da app, che sia più snello e più user-friendly rispetto a quello attualmente presente. Per soddisfare le richieste del PO (Product Owner) dovrai rispondere ai seguenti requisiti.

RF-01 — Flusso di registrazione multi-step

L'applicazione deve guidare l'utente attraverso 8 step sequenziali di inserimento dati. L'utente non può accedere a uno step successivo senza aver completato e validato quello corrente. Deve essere visibile un indicatore di progresso (es. stepper o barra).

Step 1 — Dati anagrafici

- Nome (obbligatorio, solo lettere)
- Cognome (obbligatorio, solo lettere)
- Codice fiscale (obbligatorio, formalmente valido)

Step 2 — Credenziali

- E-mail (obbligatoria, formato valido)
- Password (obbligatoria, minimo 8 caratteri, almeno una lettera maiuscola e un numero)
- Conferma password (obbligatoria, deve coincidere con la password)

Step 3 — Dati di contatto

- Numero di telefono (obbligatorio, formato numerico, minimo 9 cifre)
- Indirizzo (obbligatorio, testo libero)

Step 4 — Caricamento foto (opzionale)

- Form di caricamento di una immagine in primo piano.
- Se l'immagine non è in primo piano o è sfocata l'applicazione deve mostrare un messaggio di errore e non permettere di andare avanti.

Step 5 — Scelta repertorio

- Visualizzazione di una schermata che presenti all'utente la scelta di uno o più repertori per i quali vuole essere tutelato tra: Musica, Cinema, DOR (Dramma e Opere Radiotelevisive), Lirica e OLAF Opere Letterarie e OLAF Arti Figurative.
- La scelta del repertorio può avvenire una semplice lista di opzioni o una selezione di icone o immagini rappresentative di ciascun repertorio.

Step 6— Riepilogo e conferma

- Visualizzazione in sola lettura di tutti i dati inseriti negli step precedenti
- Pulsante "Modifica" per tornare a uno step specifico
- Pulsante "Conferma registrazione" per inviare i dati al backend

Step 7 — Pagamento (opzionale)

- L'applicazione verifica che l'utente abbia meno di 30 anni, in caso affermativo mostra un messaggio che informa della gratuità del servizio per associati under 30; altrimenti visualizza un avviso, che l'utente dovrà esplicitamente accettare, che informi l'utente sulla necessità di inoltrare il pagamento di una quota che verrà calcolata entro 30 giorni dall'iscrizione.

Step 8 — Conferma di avvenuta registrazione e conclusione del processo (opzionale)

- Visualizzazione dello stato della registrazione (success / error)
- Visualizzazione di un pulsante di redirect alla pagina di login

RF-02 — Registrazione utente (Backend)

- Se l'e-mail o il codice fiscale risultano già presenti nel database, associati allo stesso utente, l'applicazione deve mostrare a video che l'utente è già esistente e rimandare al login.

RF-03 — Login

- L'applicazione deve esporre una schermata di login con i campi e-mail e password.
- In caso di credenziali errate, viene mostrato un messaggio di errore generico (senza specificare se è sbagliata l'e-mail o la password).

Formattato: Normale, Giustificato

RF-04 — Schermata post-login (opzionale)

- Dopo il login, l'utente viene reindirizzato a una schermata di benvenuto che mostra nome, cognome, repertori scelti e la foto inserita in fase di registrazione, se presente.
- La schermata deve contenere un pulsante di logout che invalida il token in memoria e reindirizza al login.

4.3 La tecnologia — Requisiti non funzionali

Il dipartimento di Information Technology richiede che per lo sviluppo di questa nuova funzionalità vengano rispettati dei requisiti tecnici legati a tecnologie, pratiche di sicurezza e manutenibilità del prodotto. Per ottenere l'ok del Tech Lead alla pubblicazione del tuo prodotto dovrai attenerti alle seguenti richieste.

In particolare, lo stack tecnologico richiesto è il seguente:

Layer	Tecnologia
Frontend	Vue 3 (Composition API) + Capacitor
Backend	Node.js ≥ 18 + Express + TypeScript
Persistenza	File JSON locale
Autenticazione	JWT (JSON Web Token)

RNF-01 — Tecnologie obbligatorie

- Il frontend deve essere sviluppato con **Vue 3.**
- Il progetto Vue deve essere configurato con **Capacitor** (anche se la consegna avviene come web app).
- Il backend deve essere scritto in **TypeScript** con framework Express

RNF-02 — Persistenza

- I dati degli utenti devono essere salvati in un **file users.json** nella directory del backend.
- Il file deve essere letto e scritto tramite il modulo fs nativo di Node.js.
- Non è consentito l'uso di ORM, database relazionali o NoSQL.
- L'endpoint POST /api/register deve ricevere i dati dell'utente e salvarli nel file JSON.

RNF-03 — Validazione (opzionale)

- Tutta la validazione dei form deve avvenire **lato frontend** prima dell'invio.
- Il backend deve comunque eseguire una **validazione minima** dei dati ricevuti e restituire errori HTTP appropriati (400 Bad Request per dati mancanti o malformati).

RNF-04 — Sicurezza base (opzionale)

- Le password non devono mai essere trasmesse o salvate in chiaro.
- La foto dell'utente deve essere salvata come stringa base64 nel JSON di storage
- Il JWT deve essere firmato con una chiave segreta definita come variabile d'ambiente (JWT_SECRET) in un file .env.
- L'endpoint POST /api/login deve verificare le credenziali.
- In caso di successo, il backend restituisce un **JWT** con scadenza di 1 ora.
- Il token JWT deve essere salvato in memoria nell'app frontend (non in localStorage).

RNF-05 — Struttura del progetto

- Il progetto deve essere suddiviso in due cartelle distinte: /frontend e /backend.
- Il backend deve seguire una struttura minimale con separazione tra **routes**, **controllers** e **services**.
- Il frontend deve organizzare i 6 step come **componenti Vue separati**.

RNF-06 — Codice e leggibilità

- Il codice TypeScript del backend deve essere tipizzato esplicitamente (no any non giustificato).
- Le variabili e le funzioni devono avere nomi significativi in inglese.
- Ogni endpoint del backend deve essere documentato con un commento che descriva input atteso e output.

RNF-07 — Esecuzione locale

- Il progetto deve poter essere avviato localmente con i comandi npm install e npm run dev sia per il frontend che per il backend.
- Il README.md deve contenere le istruzioni di setup, le variabili d'ambiente richieste e la descrizione degli endpoint disponibili.
- Il frontend deve essere mobile responsive e deve essere navigabile da browser (obbligatorio) e da dispositivo mobile (opzionale).

RNF-08 — Unit testing

- Il codice di frontend deve essere coperto da unit test vitest con le seguenti soglie minime:
  - Branches: 70%
  - Lines: 70%
  - Functions: 70%
  - Statements: 70%

RNF-09 — UI components (opzionale)

- La libreria con i componenti grafici da importare è la @itsiae/siae-design-system. Sta a voi scoprire quali componenti sono disponibili e decidere quali utilizzare.

5. Requisiti Tecnici

Story 1 — Plugin

Requisito	Dettaglio
Righe di codice	Max 800 righe totali tra hook, script e agent. I file di test e le skill (Markdown) non contano.
Test hook/script	Almeno 6 test case funzionanti sugli hook e sugli script.
Subagent budget	Ogni agent deve dichiarare max_tokens nel frontmatter.
Installabilità	Il plugin deve essere installabile e funzionante su una macchina pulita.
Repository	Privato su GitHub. README completo.
Conventional Commits	Tutti i commit devono seguire il formato: tipo(scope): descrizione

Anti-allucinazione	Le skill devono avere step concreti e verificabili, non istruzioni vaghe.

Story 2 — SIAE+

Requisito	Dettaglio
Frontend	Vue 3 con Capacitor configurato. Mobile responsive, navigabile da browser.
Backend	TypeScript + Express. Struttura separata: routes, controllers, services.
Persistenza	File users.json con fs nativo. No ORM, no database.
Test coverage frontend	70% minimo (branches, features, lines, statements) con vitest.
Tipizzazione	TypeScript esplicito nel backend. No any non giustificato.
Esecuzione locale	npm install + npm run dev funzionanti sia per frontend che backend.
Struttura progetto	Due cartelle distinte: /frontend e /backend. Step come componenti Vue separati.

Punti bonus (entrambe le story)

- Skill per CI/CD: guida Claude nella configurazione di pipeline CI/CD
- GitHub Actions: Action nel repo che esegue i test automaticamente su push/PR
- Hook aggiuntivi funzionanti (TDD gate, post-tool logger)
- Agent aggiuntivi con budget controllato
- Validazione backend dei dati ricevuti (RNF-03)
- Sicurezza: hashing password, JWT firmato con variabile d'ambiente (RNF-04)
- Schermata post-login con logout (RF-04)

6. Criteri di Valutazione

La valutazione è sull'intero progetto. Plugin e prodotto pesano uguale.

Criterio	Peso	Cosa guardiamo
Hook funzionanti	Alto	Gli hook agiscono nei contesti corretti: bloccano, iniettano, loggano.
Qualità delle skill	Alto	4 skill obbligatorie con frontmatter, trigger precisi, step concreti.

Agent con budget	Medio	Almeno 1 agent con system prompt sensato e max_tokens dichiarato.
Test plugin	Medio	Almeno 6 test case reali sugli hook/script. Non stub vuoti.
Installabilità plugin	Alto	Il plugin funziona al primo tentativo su una macchina pulita.
Flusso registrazione	Alto	Gli step obbligatori (1-6) funzionano end-to-end con validazione frontend.
Backend funzionante	Alto	Register e login operativi. Persistenza su users.json.
Test coverage frontend	Alto	70% minimo con vitest. Test reali, non placeholder.
Coerenza plugin → prodotto	Alto	La git history di SIAE+ dimostra che il plugin era attivo.
Storia git	Medio	Commit frequenti, Conventional Commits, progressione leggibile.
README	Medio	Spiega il PERCHÉ delle scelte. Istruzioni di setup chiare.
Design SIAE+	Medio	Coerenza con la brand identity e il design system dell'app SIAE+.
Presentazione	Alto	15–20 minuti: mostra plugin e prodotto in azione insieme.
Bonus	Basso	GitHub Actions, CI/CD skill, sicurezza, validazione backend, post-login.

6.1 Livelli di Completamento

Livello	Plugin (Story 1)	Prodotto (Story 2)
Base	2 hook funzionanti + 4 skill + 1 agent. Installabile.	Step 1-6 funzionanti. Login e register operativi. Vue 3 + Express.
Intermedio	3+ hook, skill aggiuntive, agent con budget, 6+ test, README completo.	Validazione frontend completa, design coerente con SIAE+, test coverage ≥ 50%.
Avanzato	4 hook, 6+ skill, 2+ agent, test solidi, storia git pulita.	Coverage ≥ 70%, step 7-8 implementati, sicurezza base (hashing, JWT).
Eccellenza	Avanzato + GitHub Action CI + demo live del plugin attivo su SIAE+.	Avanzato + schermata post-login + validazione backend + UX curata.

Appendice A — Esempi di Riferimento

Questi sono esempi generici per capire il formato e i pattern. Non copiarli alla lettera: adattali al tuo plugin.

A.1 Esempio: Hook (hooks/hooks.json)

Il file hooks/hooks.json configura quali eventi intercettare e quali script eseguire:

```
{
"hooks": {
"SessionStart": [
{
"hooks": [
{
"type": "command",
"command": "${CLAUDE_PLUGIN_ROOT}/scripts/session-start.sh"
}
]
}
],
"PreToolUse": [
{
"matcher": "Bash",
"hooks": [
{
"type": "command",
"command": "${CLAUDE_PLUGIN_ROOT}/scripts/pre-commit.sh"
}
]
}
]
}
}
```

Lo script session-start.sh scansiona le cartelle del plugin e stampa su stdout il contesto. Claude Code cattura questo output e lo usa per tutta la sessione. Ecco un esempio:

```
#!/usr/bin/env bash
# scripts/session-start.sh
# Questo script viene eseguito automaticamente all'avvio di ogni sessione.
# Tutto ciò che stampa su stdout diventa contesto persistente per Claude.

PLUGIN_DIR="$(cd "$(dirname "$0")/.." && pwd)"

echo "=== MIO-PLUGIN ATTIVO ===="
echo ""

# Scansiona skills/ ed estrae nome e trigger dal frontmatter YAML
echo "## Skill disponibili"
for skill_dir in "$PLUGIN_DIR"/skills/*/; do
if [ -f "$skill_dir/SKILL.md" ]; then
name=$(sed -n "s/^name: *//p" "$skill_dir/SKILL.md" | head -1)
# Estrai la prima riga di TRIGGER dalla description
trigger=$(grep "TRIGGER:" "$skill_dir/SKILL.md" | head -1 | sed "s/.*TRIGGER: *//")
echo "  - $name: $trigger"
fi
done
# Scansiona agents/ ed estrae nome e budget
echo ""
echo "## Agent disponibili"
for agent in "$PLUGIN_DIR"/agents/*.md; do
if [ -f "$agent" ]; then
name=$(sed -n "s/^name: *//p" "$agent" | head -1)
budget=$(sed -n "s/^max_tokens: *//p" "$agent" | head -1)
echo "  - $name (budget: ${budget:-non specificato} tokens)"
fi
done

echo ""
```

```
echo "## Regole attive"
echo "  - I commit devono seguire Conventional Commits (validati da hook)"
echo "  - Le skill obbligatorie non si bypassano"
echo ""
echo "Usa /brainstorming prima di implementare. Usa /verification prima di dichiarare fatto."
```

Lo script pre-commit.sh riceve JSON su stdin. Ecco un esempio:

```
#!/usr/bin/env bash
set -euo pipefail

INPUT=$(cat)
CMD=$(echo "$INPUT" | jq -r '.tool_input.command // ""')

# Ignora se non è un git commit
echo "$CMD" | grep -q "git commit" || exit 0

# Estrai il messaggio di commit
MSG=$(echo "$CMD" | grep -oP '(?<=-m ["'])[^"']+' || true)
[ -z "$MSG" ] && exit 0

# Valida Conventional Commits
REGEX="^(feat|fix|docs|style|refactor|test|chore|perf|ci|build|revert)(\(.+\))?: .{1,72}$"
if ! echo "$MSG" | grep -Eq "$REGEX"; then
echo "Commit bloccato: messaggio non valido." >&2
echo "Formato richiesto: tipo(scope): descrizione" >&2
echo "Tipi validi: feat fix docs style refactor test chore perf ci build revert" >&2
exit 2
fi
exit 0
```

A.2 Esempio: Skill (skills/brainstorming/SKILL.md)

```
---
```

```
name: brainstorming
description: |
TRIGGER: usa questa skill quando l'utente chiede di implementare,
creare, scrivere o aggiungere una feature non banale.
TRIGGER: usa quando l'utente dice "come faccio a..." per un
problema che richiede più di 10 righe di codice.
NON usare per fix di 1-2 righe o modifiche di configurazione.
---

# 🧠 Brainstorming

## Legge di Ferro
Non scrivere codice finché non hai un piano.

## Step
1. **Definisci il problema** in una frase.
2. **Elenca i casi limite** (almeno 3).
3. **Valuta 2 approcci alternativi** con pro e contro.
4. **Stima la complessità**: banale / media / alta.
5. **Decidi**: se alta, scrivi un piano in docs/plans/ prima di procedere.

## Rischio: MEDIUM
```

A.3 Esempio: Agent (agents/reviewer.md)

```
---
name: reviewer
description: Revisiona codice per qualità, bug e aderenza agli standard.
model: sonnet
max_tokens: 8000
tools:
- Read
```

```
- Grep
- Glob
---

Sei un code reviewer esperto. Ricevi file o diff e produci un report.

## Regole
- Analizza SOLO il codice passato, non fare assunzioni.
- Classifica ogni finding: CRITICAL / WARNING / INFO.
- Per ogni finding: problema, suggerimento fix, file e riga.
- Se non trovi problemi, dillo esplicitamente.

## Output
```
Review Report
CRITICAL
- [file:riga] Problema → Fix
WARNING
- [file:riga] Problema → Fix
INFO
- [file:riga] Nota
Verdict: PASS / FAIL
```
```

A.4 Esempio: plugin.json

```
{
"name": "mio-plugin",
"version": "0.1.0",
"description": "Plugin per disciplinare il ciclo di sviluppo con Claude Code",
"author": "Il Tuo Nome",
"homepage": "https://github.com/username/mio-plugin"
}
```

A.5 Esempio: Test di un Hook

I test verificano che gli hook funzionino. Ecco un test bash minimale:

```
#!/usr/bin/env bash
# tests/test-pre-commit.sh
set -euo pipefail
HOOK="./scripts/pre-commit.sh"
PASS=0; FAIL=0

run_test() {
local desc="$1" input="$2" expected="$3"
actual=0
echo "$input" | bash "$HOOK" > /dev/null 2>&1 || actual=$?
if [ "$actual" -eq "$expected" ]; then
echo "✅ $desc"; ((PASS++))
else
echo "❌ $desc (expected $expected, got $actual)"; ((FAIL++))
fi
}

# Commit valido → pass (exit 0)
run_test "valid commit" \
'{"tool_name":"Bash","tool_input":{"command":"git commit -m \"feat(auth): add login\""}}' 0

# Commit invalido → block (exit 2)
run_test "invalid commit" \
'{"tool_name":"Bash","tool_input":{"command":"git commit -m \"fixed stuff\""}}' 2

# Non è un commit → pass (exit 0)
run_test "non-commit" \
'{"tool_name":"Bash","tool_input":{"command":"ls -la"}}' 0
```

```
echo ""; echo "Results: $PASS passed, $FAIL failed"
[ "$FAIL" -eq 0 ] || exit 1
```

A.6 Esempio: GitHub Action (Bonus)

Se hai tempo, aggiungi una Action che esegue i test su ogni push:

```
# .github/workflows/test.yml
name: Test Plugin
on: [push, pull_request]
jobs:
test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- name: Install jq
run: sudo apt-get install -y jq
- name: Run hook tests
run: |
chmod +x scripts/*.sh tests/*.sh
bash tests/test-pre-commit.sh
```